Student Management Portal

Project Requirements & Technical Specification

A Practice Project for Junior Developers

Tech Stack	MERN (MongoDB, Express.js, React, Node.js)
Difficulty Level	Intermediate
Estimated Duration	2 Weeks
Target Audience	Junior Developers
Date	7th April 2026

1. Project Overview

You are tasked with building a Student Management Portal — a full-stack web application using the MERN stack. This project will test your ability to design, build, and connect a real-world application from scratch, covering both backend API development and frontend UI work.

The system serves two distinct types of users:

- Students — who can log in, view their personal profile, and check marksheets uploaded by their faculty.
- Faculty Members — who can log in, manage students under their mentorship, upload marksheets, and edit student records.

This project is intentionally scoped to be challenging but achievable. Read this document thoroughly before writing a single line of code.

2. Learning Objectives

By completing this project, you will demonstrate your understanding of:

- Designing and building a RESTful API with Node.js and Express.js
- Structuring a MongoDB database with Mongoose schemas and relationships
- Implementing JWT-based authentication and role-based access control (RBAC)
- Building a multi-page React frontend with protected routes
- Handling file uploads (PDF/image) on both the backend and frontend

- Writing clean, modular, and well-organised code

3. Technology Stack

3.1 Backend

- Runtime: Node.js
- Framework: Express.js
- Database: MongoDB (via Mongoose ODM)
- Authentication: JSON Web Tokens (JWT)
- File Uploads: Multer
- Password Hashing: bcrypt

3.2 Frontend

- Library: React (functional components + Hooks)
- Routing: React Router DOM
- HTTP Client: Axios
- Styling: Your choice (CSS Modules, Tailwind CSS, or Bootstrap)

3.3 Dev Tools

- Version Control: Git & GitHub (mandatory — commit regularly)
- API Testing: Postman
- Environment Variables: dotenv
- Package Manager: npm

4. System Roles & Permissions

The application has two user roles. Every feature must respect these boundaries — a student must never be able to access faculty-only endpoints, and vice versa.

Feature / Action	Who Can Perform It
Login	Both Students and Faculty
View own profile	Both Students and Faculty
View own marksheet	Students only
View student list	Faculty only
View a student's details	Faculty only (assigned students)
Upload marksheet for student	Faculty only

Feature / Action	Who Can Perform It
Edit student details	Faculty only
Add a new student	Faculty only
Delete a student record	Faculty only

5. Detailed Feature Requirements

5.1 Authentication

Both students and faculty share a single login page. The role (student or faculty) must be determined from the database after verifying credentials, not from a dropdown or toggle on the form.

Requirements:

- Passwords must be hashed using bcrypt before storing in MongoDB.
- On successful login, return a JWT token. Store it on the frontend (localStorage or a cookie).
- Every protected API route must validate the JWT token via middleware.
- Different roles must redirect to different dashboards after login.
- Logging out must clear the token from the frontend.

5.2 Student Features

5.2.1 View Profile

- Students can view their own personal details: full name, email address, roll number, department, and year of study.
- Students cannot edit their own details — only faculty can do that.

5.2.2 View Marksheet

- Students can view the marksheet that has been uploaded for them by their assigned faculty.
- The marksheet may be a PDF or an image file.
- If no marksheet has been uploaded yet, display a friendly message (e.g., 'No marksheet uploaded yet').
- Students can only view their own marksheet — never another student's.

5.3 Faculty Features

5.3.1 Dashboard

- Faculty members see a dashboard listing all students assigned to them.
- Each entry should show the student's name, roll number, and department.

- The list should be searchable or filterable by name or roll number.

5.3.2 View Student Details

- Faculty can click on any student in their list to view that student's full profile.
- Faculty can only view students assigned to them, not all students in the database.

5.3.3 Add Student

- Faculty can create a new student record with the following fields:
  - Full Name (required)
  - Email Address (required, must be unique)
  - Roll Number (required, must be unique)
  - Department (required)
  - Year of Study (required — 1st, 2nd, 3rd, or 4th)
  - Password (required — must be hashed before saving)
- The new student is automatically assigned to the faculty member who created them.

5.3.4 Edit Student Details

- Faculty can update any field on a student's profile (except roll number and email once set, unless you choose to allow it).
- The form should pre-fill with existing data.
- Only the faculty member assigned to that student can edit their record.

5.3.5 Upload Marksheet

- Faculty can upload a marksheet file (PDF or image) for any student assigned to them.
- File must be stored on the server using Multer.
- If a marksheet already exists, uploading a new one should replace the old file.
- Store the file path or filename in the student's database record.

5.3.6 Delete Student

- Faculty can permanently delete a student record.
- Show a confirmation prompt before deletion.
- Deleting a student should also remove their associated marksheet file from the server.

6. Database Design

You will have at minimum two collections in MongoDB. You may add more if your design requires it.

6.1 User Collection

One collection can store both students and faculty using a role field to differentiate them. Alternatively, you can use two separate collections — the choice is yours, but justify it in your README.

Field	Type	Notes
name	String	Required
email	String	Required, unique
password	String	Required, bcrypt hashed
role	String	'student' or 'faculty'
rollNumber	String	Required for students, unique
department	String	Required
yearOfStudy	Number	1–4, students only
assignedFaculty	ObjectId	Ref: User (faculty), students only
marksheet	String	File path/name, students only

7. API Reference

All API routes should be prefixed with /api. Protected routes must include a valid JWT token in the Authorisation header (<token>).

7.1 Auth Routes

Method	Endpoint	Description
POST	/api/auth/login	Log in for both students and faculty
POST	/api/auth/logout	Invalidate session / clear token

7.2 Student Routes (Faculty Only)

Method	Endpoint	Description
GET	/api/students	Get all students assigned to logged-in faculty
GET	/api/students/:id	Get a single student's details
POST	/api/students	Create a new student
PUT	/api/students/:id	Update a student's details
DELETE	/api/students/:id	Delete a student record
POST	/api/students/:id/marksheet	Upload or replace a student's marksheet

7.3 Student Routes (Student Only)

Method	Endpoint	Description
GET	/api/me	Get the logged-in student's own profile
GET	/api/me/marksheet	Download or view own marksheet

8. Frontend Pages & Components

Below is the minimum set of pages your React application must include. You may add more as you see fit.

Page	Purpose
/login	Single login page for both roles
/student/dashboard	Student's home — shows profile summary
/student/marksheet	View the uploaded marksheet
/faculty/dashboard	Faculty home — list of assigned students
/faculty/students/:id	View a single student's full profile
/faculty/students/new	Add a new student form
/faculty/students/:id/edit	Edit student details form
/faculty/students/:id/marksheet	Upload marksheet form

Important: All student routes and faculty routes must be protected. A student should not be able to navigate to a faculty page, and an unauthenticated user should be redirected to /login.

9. Technical Requirements & Rules

The following are non-negotiable requirements for this project:

1. Never store plain-text passwords. All passwords must be hashed with bcrypt.
2. Never expose your JWT secret or MongoDB connection string in your code. Use a .env file and add it to .gitignore.
3. All protected routes must be guarded by an Express middleware that validates the JWT token.
4. Role-based access must be enforced on the backend — do not rely only on the frontend to restrict access.

5. All API responses must return meaningful HTTP status codes (200, 201, 400, 401, 403, 404, 500).
6. All forms must include basic client-side and server-side validation (e.g., required fields, valid email format).
7. Error messages returned from the API should be descriptive but not expose sensitive system information.
8. Your GitHub repository must have a README.md explaining how to set up and run the project locally.

10. Bonus Challenges

Completed all the requirements and want to go further? Try these:

- Pagination: Add pagination to the faculty's student list (e.g., 10 students per page).
- Search: Add a live search bar on the faculty dashboard to filter students by name or roll number.
- Email Notifications: Send a welcome email to a new student when they are added.
- Audit Log: Keep a log of when marksheets are uploaded and by whom.
- Dark Mode: Add a dark/light mode toggle to the frontend.
- Responsive Design: Ensure the UI works well on mobile screens.
- Deployment: Deploy the backend to Render or Railway, and the frontend to Vercel.

11. Submission Checklist

Before submitting your project, confirm the following:

- GitHub repository is public, and the link has been shared.
- README.md contains setup instructions, environment variable list, and a brief description.
- .env file is NOT committed to the repo — it is in .gitignore.
- All six milestones are completed and working.
- Login works correctly for both student and faculty roles.
- Faculty can perform all CRUD operations on students.
- Marksheet upload and display both work end-to-end.
- Protected routes reject unauthenticated and unauthorised access.
- No console errors appear in the browser or terminal during normal use.
- The app runs successfully after following your README setup steps.

Good luck — take it one phase at a time and ask questions when you're stuck.

You've got this.